HDOC-64

Finding the Duplicate Element in a Variable-Length Array Using Pointers

Problem Statement

Write a C program that reads the size and elements of a variable-length integer array, nums[]. The array contains exactly one element repeated once, while every other element appears exactly once. Pass the array to a function named duplicate_element() using its address and use pointers to access the array. The function must find and return the repeated element efficiently using the XOR technique.

Algorithm / Logic

1. Read the array size n. Since exactly one value is repeated once, the array contains n elements but only n-1 distinct values.
2. Read the n elements into the variable-length array nums[n].
3. Pass nums (the address of its first element) and n to duplicate_element().
4. Inside the function, XOR all array elements using pointer notation: result ^= *(nums + i).
5. XOR the expected distinct values from 1 through n-1 into the same result.
6. Because x XOR x = 0 and x XOR 0 = x, all non-repeated values cancel and the duplicate remains.
7. Return and display the duplicate element.

C Program

```
#include <stdio.h>

int duplicate_element(int *nums, int n)
{
    int result = 0;
    int i;

    /* XOR all elements of the array using pointer notation */
    for (i = 0; i < n; i++)
        result ^= *(nums + i);

    /* XOR all expected distinct values: 1 to n - 1 */
    for (i = 1; i <= n - 1; i++)
        result ^= i;

    return result;
}

int main(void)
{
    int n, i;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    if (n < 2) {
        printf("Array size must be at least 2.\n");
        return 1;
    }

    int nums[n];

    printf("Enter %d elements (values from 1 to %d):\n", n, n - 1);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    int duplicate = duplicate_element(nums, n);
```

```
    printf("Repeated element = %d\n", duplicate);

    return 0;
}
```

Explanation

The array is passed to duplicate_element() through the pointer int *nums. In C, the array name nums represents the address of the first element, so calling duplicate_element(nums, n) passes the array address without copying the whole array. The expression *(nums + i) accesses the same element as nums[i].

The XOR method is efficient because equal values cancel each other. For example, if nums = {1, 3, 4, 2, 2}, XORing all array values and then XORing 1, 2, 3, and 4 leaves only 2. The algorithm uses O(n) time and O(1) extra space.

Assumption: The distinct values are integers from 1 to n-1, with exactly one of them occurring twice. This is the condition that makes the single-pass XOR approach valid.

Test Cases

S.No.	n	Array Elements	Duplicate
1	5	1 3 4 2 2	2
2	6	1 5 3 4 2 3	3
3	4	1 2 3 1	1
4	7	6 1 4 2 5 3 6	6

Sample Output

```
Enter the size of the array: 5
Enter 5 elements (values from 1 to 4):
1 3 4 2 2
Repeated element = 2
```

Result

The program successfully finds and displays the only repeated element in the variable-length integer array. The array is passed by address using pointers, and the XOR technique provides an O(n) time and O(1) extra-space solution.